- [ ] Data Center Management:
  - [ ] Data Center Architecture, Requirements & prerequisites
  - [ ] Required Physical Area for Equipment and Unoccupied Space
  - [ ] Required power to run all the devices
  - [ ] Required cooling and HVAC
  - [ ] Required Network bandwidth
  - [ ] Selecting a Geographic Location
  - [ ] Guidelines for Planning a Data Center
- [ ] Infrastructure in a Data Center
  - [ ] Modular Cabling Design
  - [ ] Points of Distribution
  - [ ] ISP Network Infrastructure and WAN Links
  - [ ] Network Operations Center and Monitoring
  - [ ] Data center physical security, Logical security, and cleaning
- [ ] Basics of Virtualization
  - [ ] Introduction of virtualization
  - [ ] Virtualization types: type1, type2
  - [ ] Virtualization, Hardware Virtualization, Para-Virtualization, Cloning, Snapshot, and Template
  - [ ] Storage Area Network
  - [ ] Configuring a SAN (FreeNAS)
  - [ ] Using SAN for high availability
  - [ ] ZFS Volume Configuration
  - [ ] IP-Based Storage Communication
- [ ] Cloud Computing
- [ ] Introduction to Cloud
- [ ] Cloud Computing, Cloud SPI Model, Cloud Computing Deployment Model (Public, Private, and Hybrid), Cloud Security (SLA and IAM).
- [ ] Cloud architecture
- [ ] Service models: IaaS, PaaS, SaaS
- [ ] Services provided by Cloud (Compute, Database, Developer Tools, Storage, Media, Mobile, Web, Security, Integration, etc.)
- [ ] Cloud development best practices
- [ ] Learn how cloud API integration is done
- [ ] Infrastructure Provisioning using Chef and PuppetChef Basics

- [ ] Chef Architecture
- [ ] Chef installation
- [ ] Chef Usage
- [ ] Puppet Basics
- [ ] Puppet Architecture
- [ ] Puppet Installation
- [ ] Puppet Usage
- [ ] Configuring Chef to perform infrastructure provisioning
- [ ] Configuring Puppet to perform infrastructure provisioning

- [ ] Network Monitoring using Prometheus
  - [ ] Prometheus and its concepts
  - [ ] Prometheus installation
  - [ ] Prometheus usage
  - [ ] Prometheus monitoring
  - [ ] Install and configure Prometheus on a Linux server
  - [ ] Monitor Linux systems using Node Exporter
  - [ ] Monitor Windows systems using Windows Exporter
  - [ ] Visualize system metrics using Prometheus Web UI

- [ ] DevOps & Docker
  - [ ] Introduction to DevOps
  - [ ] CI/CD Pipelines
  - [ ] Docker concepts
  - [ ] Docker commands
  - [ ] Docker composes
  - [ ] Docker Networking
  - [ ] Volumes
  - [ ] Docker Compose
  - [ ] Tagging and pushing images to Docker hub
  - [ ] Create an httpd container running on port 8000. Name the container as web5. Do not map any directory. Create an “index.html” file & copy this file inside the container and check if the website displays your webpage. Again, modify the “index.html” file and copy it inside the container, and check if the website is updated or not
  - [ ] Create a C program(exe). Test the program on-base machine. Then create an image using Docker file to execute this exe program. Give image name as “Name: v1”
  - [ ] Try the above with a program that requires user input

- [ ] Git and GitHub
  - [ ] Introduction
  - [ ] Introduction to Git
  - [ ] Core Concepts
  - [ ] Going Command Line
  - [ ] Basic Git workflow with GitHub
  - [ ] Welcome to GitHub
  - [ ] Setup the Project Folder
  - [ ] Git Configuration (User Name and Email)
  - [ ] Copy the Repository from GitHub to Your Local Computer (git clone)
  - [ ] The First Commit
  - [ ] Publishing Changes back to GitHub (push)
- [ ] Jenkins
  - [ ] Introduction of Jenkins
  - [ ] Installing Jenkins
  - [ ] Creating simple jobs
  - [ ] CI/CD pipeline using Jenkins
  - [ ] Integrating Jenkins with GitHub
  - [ ] Integrating Jenkins with Docker
  - [ ] Create a C program to take input from the user and test it. Then create an image using Docker file. Test image by the running container. Then push the image to the docker hub repository. Then also export the image in a “cprog-back.tar” file
  - [ ] Create a CI/CD pipeline using Jenkins, Docker, and GitHub to automate the web by using the NGINX image
  - [ ] Create a CI/CD pipeline using Jenkins, Docker, and GitHub to automate the web by using the httpd image
- [ ] AWS
  - [ ] Introduction to AWS
  - [ ] Services provided by AWS: EC2, Lambda, S3
  - [ ] Introduction Virtual Private Cloud (VPC) Setup
  - [ ] Create AWS EC2 instance
  - [ ] Create AWS Lambda
  - [ ] Create AWS S3 bucket
  - [ ] Create AWS VPC

- [ ] Create a new VPC by the name ditiss-lab with public & private subnet. Assign network address as 172.20.0.0/16. Assign 172.20.5.0/24 to the public and 172.20.10.0/24 to the private subnet. Create one instance and connect to a private subnet. Create one instance and connect to the public subnet. Install httpd on the private instance and check with curl from the public connected instance

- [ ] Container Orchestration
  - [ ] Container Orchestration: Kubernetes, Docker swarm
  - [ ] Micro Service Deployment
  - [ ] Create a new docker image using NGINX. Push it to docker hub. Create a service on docker swarm. Keep initial replicas as 1 and then increase the replicas to 10. Reduce the replicas to 2. Create a new image by modifying “index.html”. Push to docker hub and then update the service for a new image
  - [ ] Do the above using Kubernetes
  - [ ] Configure a Linux as a Master node using Kubernetes and make 2 to 3 worker node clusters

- [ ] Ansible
  - [ ] Introduction to Ansible and Configuration Management
  - [ ] Setting Up Ansible Environment
  - [ ] Ansible Playbooks and YAML Basics
  - [ ] Managing Ansible Inventory
  - [ ] Ansible Roles and Reusability
  - [ ] Setting Up Ansible Environment
  - [ ] Install Ansible on a local machine or a virtual machine
  - [ ] Configure SSH keys for password less authentication to a target node
  - [ ] Verify the setup by running the ansible -m ping all command
  - [ ] Writing and Executing Ansible Playbooks
  - [ ] Create a simple Ansible playbook to install and start a webserver (e.g., Apache or Nginx) on a target node
  - [ ] Use the playbook to ensure the web server is running and
  - [ ] serving a default webpage
  - [ ] Verify the setup by accessing the webpage from a browser
  - [ ] Managing Ansible Inventory and Roles
  - [ ] Create a static inventory file listing multiple target node
  - [ ] Develop a role to install and configure a database server (e.g., MySQL or PostgreSQL)
  - [ ] Apply the role using an Ansible playbook to deploy the database server on one of the target nodes listed in the inventory

- [ ] Terraform

- [ ] Introduction to Infrastructure as Code (IaC) and Terraform
- [ ] Setting Up the Terraform Environment
- [ ] Writing and Organizing Terraform Configuration Files
- [ ] Terraform State Management
- [ ] Terraform Modules and Reusability
- [ ] Setting Up the Terraform Environment
- [ ] Install Terraform on a local machine
- [ ] Configure AWS credentials for Terraform to interact with AWS services
- [ ] Create and apply a basic Terraform configuration to provision instance in AWS
- [ ] Writing and Organizing Terraform Configuration Files
- [ ] Write a Terraform configuration file to provision a VPC, subnet, nd an EC2 instance within the subnet
- [ ] Use variables and outputs to make the configuration more ynamic and reusable
- [ ] Apply the configuration and verify the resources are created in WS
- [ ] Terraform State Management and Modules
- [ ] Create a Terraform module for provisioning an EC2 instance with a security group
- [ ] Use the module in a Terraform configuration to provision multiple EC2 instances in different subnets
- [ ] Manage the Terraform state using a remote backend (e.g., AWS S3) to enable collaboration and state sharing